MACHINE LEARNING

Module Code: CSC83101

LECTURE

TUYISHIMIRE Joseph

tuyishimirejo1992@gmail.com

UNIVERSITY Of KIGALI

NUMPY FOR MACHINE LEARNING

Definition

NumPy stands for **Numerical Python**.
It is a **Python library** used for:

- Numerical calculations
- Working with large datasets
- Mathematical operations in Machine Learning
- NumPy is the **foundation of almost all ML libraries** like Pandas, Scikit-learn, TensorFlow.

UNIVERSITY Of KIGALI

Why NumPy is Important in Machine Learning

Machine Learning works mainly with:

- Numbers
- Vectors
- Matrices

NumPy provides:

- Fast performance
- Efficient memory usage
- Mathematical functions

Importing NumPy Syntax

```
import numpy as np
```

Explanation

numpy → library name

as np → alias (short name)

np is commonly used in ML code

NumPy Array

Definition

- A NumPy array is a **collection of elements of the same data type**, stored in a continuous memory block.

- It is **faster and more efficient** than Python lists.

Creating a NumPy Array Syntax

```
np.array(object)
```

Example

```
import numpy as np
arr = np.array([1, 2, 3, 4])
print(arr)
```

One-Dimensional (1D) Array Definition

- A 1D array is a **single row of elements**

Example

```
a = np.array([10, 20, 30])
Print(a)
```

Two-Dimensional (2D) Array Definition

- A 2D array represents **rows and columns** (matrix).

```
b = np.array([[1, 2, 3],
              [4, 5, 6]])
print(b)
```

Practice Section

Practice 1

- Create a NumPy array of numbers 1–20
- Find mean and standard deviation

Practice 2

- Create 3×3 matrix
- Find row-wise sum

PANDAS FOR MACHINE LEARNING

Pandas is a **Python library used for data manipulation and data analysis**.
It allows us to:

- Load datasets
- Clean data
- Analyze data
- Prepare data for Machine Learning models
- Pandas works **on top of NumPy**, which means it uses NumPy arrays internally.

UNIVERSITY Of KIGALI

Why Pandas is Important in Machine Learning

In real ML projects:

- Data is messy
- Data comes from CSV, Excel, databases
- Data contains missing values and errors

Pandas helps:

- Understand the data
- Clean the data
- Convert data into ML-ready format

import pandas as pd

pandas → library name

pd → commonly used short name

Core Data Structures in Pandas

Pandas has **two main data structures**:

- **Series** → 1-dimensional data
- **DataFrame** → 2-dimensional data (rows & columns)

Pandas Series

- **Definition**

A Series is a **one-dimensional labeled array** capable of holding:

- Numbers
- Strings
- Boolean values

Each value in a Series has:

- A value
- An inde

Creating a Series Syntax

- pd.Series(data)

```
import pandas as pd
s = pd.Series([10, 20, 30, 40])
print(s)
```

Output

```go
0    10
1    20
2    30
3    40
dtype: int64
```

Explanation

- Left side → index
- Right side → value

Custom Index Example

```
s = pd.Series([100, 200, 300], index=['a', 'b', 'c'])
print(s)
```

Pandas DataFrame

A DataFrame is a **2-dimensional labeled data structure** with:

- Rows
- Columns

Think of a DataFrame as:

- An Excel sheet
- A table in a database

Creating DataFrame from Dictionary Syntax

Pd.DataFrame(data)

Example

```python
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Salary': [50000, 60000, 70000]
}

df = pd.DataFrame(data)
print(df)
```

Explanation

- Keys → column names
- Values → column data

```
      Name  Age  Salary
0    Alice   25   50000
1      Bob   30   60000
2  Charlie   35   70000
```

Loading Data from CSV File

- Most real ML datasets are stored in **CSV files**

Syntax

```python
pd.read_csv("file_name.csv")
```

Example

```python
df = pd.read_csv("data.csv")
```

7. Viewing Data

head() Function

Shows first 5 rows by default.

```python
df.head()
```

tail() Function

Shows last 5 rows.

```python
df.tail()
```

shape Attribute

```python
df.shape
```

Output:

```pgsql
(rows, columns)
```

describe() Method

```python
df.describe()
```

Gives:

- Mean
- Standard deviation
- Min & max values

9. Accessing Columns

Access Single Column

```python
df['Age']
```

Access Multiple Columns

```python
df[['Age', 'Salary']]
```

10. Accessing Rows

iloc (Index-based)

```python
df.iloc[0]
```

iloc with range

```python
df.iloc[0:3]
```

11. Filtering Data

Condition-based Filtering

```python
df[df['Age'] > 30]
```

Multiple Conditions

```python
df[(df['Age'] > 25) & (df['Salary'] > 60000)]
```

What are Missing Values?

Missing values are data points that are absent for a specific variable in a dataset. They can be represented in various ways, such as blank cells, null values, or special symbols like "NA" or "unknown."

These missing data points pose a significant challenge in data analysis and can lead to inaccurate or biased results.

	School ID	Name	Address	City	Subject	Marks	Rank	Grade
0	101.0	Alice	123 Main St	Los Angeles	Math	85.0	2	B
1	102.0	Bob	456 Oak Ave	New York	English	92.0	1	A
2	103.0	Charlie	789 Pine Ln	Houston	Science	78.0	4	C
3	NaN	David	101 Elm St	Los Angeles	Math	89.0	3	B
4	105.0	Eva	NaN	Miami	History	NaN	8	D
5	106.0	Frank	222 Maple Rd	NaN	Math	95.0	1	A
6	107.0	Grace	444 Cedar Blvd	Houston	Science	80.0	5	C
7	108.0	Henry	555 Birch Dr	New York	English	88.0	3	B

There are many reasons for a dataset to contain missing values-:

- Due to technical issues.
- If the data comes from a survey then many people can leave blank response which can lead to missing values in the data.
- Data processing issues, privacy concerns etc.

Types of Missing Values

Missing Completely at Random (MCAR)

MCAR is a specific type of missing data in which the probability of a data point being missing is entirely random and independent of any other variable in the dataset. In simpler terms, whether a value is missing or not has nothing to do with the values of other variables or the characteristics of the data point itself.

Missing at Random (MAR)

MAR is a type of missing data where the probability of a data point missing depends on the values of other variables in the dataset, but not on the missing variable itself. For example, if someone lost a schedule, then it may be replaced by a schedule taking at random from the set of filled schedules

Missing not at random (MNAR)

MNAR is the most challenging type of missing data to deal with. It occurs when the probability of a data point being missing is related to the missing value itself.

This means that the reason for the missing data is informative and directly associated with the variable that is missing. For example, when smoking status is not recorded in patients admitted as an emergency, who are also more likely to have worse outcomes from surgery.

UNIVERSITY Of KIGALI

How missing values impact our dataset?

- It can reduce the size of the sample or dataset.
- Lack of information. If the dataset has large amount of missing values then there are high chances of lacking useful information.
- If the missing data is not handled properly, it can bias (model not properly training on dataset) the results of your analysis.
- Some statistical techniques require complete data for all variables, making them inapplicable when missing values are present.

UNIVERSITY Of KIGALI

Identify missing values

There are different methods in Python's pandas library to identify missing values.

- .isnull() -: Identifies missing values in a Series or DataFrame.
- .notnull() -: Check for missing values in a pandas Series or DataFrame. It returns a boolean Series or DataFrame, where True indicates non-missing values and False indicates missing values.
- .isna() -: Similar to notnull() but returns True for missing values and False for non-missing values

12. Handling Missing Values

Checking Missing Values

```python
df.isnull()
```

Count Missing Values

```python
df.isnull().sum()
```

Treating Missing Values

There are various techniques used to treat missing values in a dataset.

1. Remove all the missing data

- If the dataset doesn't contain significant amount of missing data then it is worthful to remove all the missing data.

The method used in Python is-:

- dropna() -: Drops rows or columns containing missing values based on custom criteria.

```python
# Drop rows where 'embarked' or 'embark_town' are missing
df.dropna(subset=['embarked', 'embark_town'], inplace=True)
# Alternatively, to drop all rows with *any* missing value (use with caution):
# df.dropna(inplace=True)
```

2. Imputation

Imputation means replacing a missing value with another value based on reasonable estimate. This have chances to give high bias.

Some common Imputation methods are -:

- Mean Imputation -: Replace missing values with the mean of the relevant variable.
- The strategy can highly be affected by outliers. Implementation -: Method 1-: **df[column_name].fillna(df[column_name].mean( ))**

Using SimpleImputer()-:

- It is defined in sklearn library. It replace missing values using a descriptive statistic (e.g. mean, median, or most frequent) along each column, or using a constant value.

```
>>> import numpy as np
>>> from sklearn.impute import SimpleImputer
>>> imp_mean = SimpleImputer(missing_values=np.nan, strategy='mean')
>>> imp_mean.fit([[7, 2, 3], [4, np.nan, 6], [10, 5, 9]])
SimpleImputer()
>>> X = [[np.nan, 2, 3], [4, np.nan, 6], [10, np.nan, 9]]
>>> print(imp_mean.transform(X))
[[ 7.   2.   3. ]
 [ 4.   3.5  6. ]
 [10.   3.5  9. ]]
```

UNIVERSITY Of KIGALI

- Here we have imported numpy and SimpleImputor and then created an instance of SimpleImputer named as imp_mean which replaces missing value (np.nan) by mean (strategy="mean").
- Then we have fitted the data to imputer and transformed it.
- We can use different strategies to impute missing values here.

Median Imputation

Median Imputation -: Replace missing values with the median of the relevant variable.

- Implementation -:

**df[column_name].fillna(df[column_name].median(
))**

- We can also use SimpleImputer all we need to do is to give strategy="median".

Mode Imputation

Mode Imputation -: Replace missing values with the mode of the relevant variable.

- Implementation -:

df[column_name].fillna(df[column_name].mode())

We can also use SimpleImputer all we need to do is to give strategy="most_frequent".

- This strategy can be challenging in case of multimodal data (having more than one mode).

Drop Missing Values

```python
df.dropna()
```

Fill Missing Values

```python
df.fillna(df.mean())
```

ML Tip

Never train ML models with missing values.

```
7]: print(df.isnull().sum())
```

```
survived         0
pclass           0
sex              0
age            177
sibsp            0
parch            0
fare             0
embarked         2
class            0
who              0
adult_male       0
deck           688
embark_town      2
alive            0
alone            0
dtype: int64
```

Dealing with missing data is a crucial step in data preprocessing.

Based on your image, the age column has 177 missing entries, while deck has 688 and embarked/embark_town each have 2.

Add New Column

```python
df['Bonus'] = df['Salary'] * 0.1
```

Rename Columns

```python
df.rename(columns={'Salary': 'Income'}, inplace=True)
```

Delete Column

```python
df.drop('Bonus', axis=1, inplace=True)
```

14. Data Type Conversion

Convert Column Type

```python
df['Age'] = df['Age'].astype(int)
```

ML Use

- Convert strings to numbers
- Prepare data for models

15. Sorting Data

Sort by Column

```python
df.sort_values('Age')
```

Descending Order

```python
df.sort_values('Age', ascending=False)
```

16. GroupBy Operation

Definition

GroupBy is used to **group data and apply aggregation**.

Example

```python
df.groupby('Age')['Salary'].mean()
```

ML Use

- Feature analysis
- Pattern detection

17. Applying Functions

apply() Method

```python
df['Salary'] = df['Salary'].apply(lambda x: x * 1.1)
```

Explanation

- Applies function to each value
- Useful for feature transformation

DATA VISUALIZATION USING MATPLOTLIB & SEABORN

- **Data Visualization?**

Data visualization is the process of **representing data in graphical form** so that:

- Patterns become visible
- Trends are easy to understand
- Outliers can be detected
- Decisions can be made easily

In Machine Learning, visualization is used **before training a model** to:

- Understand data distribution
- Detect missing values and outliers
- Choose correct preprocessing steps

- **Understanding Data Distribution:** Tools like histograms, density plots, and box plots reveal if features are normally distributed, skewed, or multimodal, which informs whether you need to apply transformations like log-scaling or normalization [1, 2].
- **Detecting Missing Values and Outliers:** Heatmaps (such as those from the Seaborn library) can pinpoint gaps in datasets, while scatter plots and box plots highlight extreme values (outliers) that might skew model gradients or lead to poor generalization [2, 3].
- **Choosing Correct Preprocessing:** Visualization identifies feature correlations (via correlation matrices) and scale differences, helping practitioners decide on feature selection, dimensionality reduction (like PCA), or specific encoding techniques for categorical variables [1, 3].

Why Data Visualization is Important in Machine Learning

Machine Learning models **do not tell you what is wrong with data**.
Visualization helps you **see problems before they cause errors**. Visualization helps to:

- Identify skewed data
- Detect outliers
- Understand feature relationships
- Choose feature transformations

UNIVERSITY Of KIGALI

What it means:

Skewed data is when most values cluster on one side, causing imbalance. This can affect model fairness and accuracy.

- **Example:**
  Imagine a histogram of customer ages where most are between 20-40, but a few very old ages stretch the range. Visualization shows this imbalance clearly.

Detect Outliers

- **What it means:**
  Outliers are data points that differ significantly from others and can distort model training.
- **Example:**
  A box plot of transaction amounts can instantly reveal unusually large or small amounts.

- Suppose you have a dataset of customer transactions with a column TransactionAmount. You want to see if there are unusually large or small transactions that could affect your analysis or model.

- import pandas as pd
- import matplotlib.pyplot as plt

- # Sample transaction data with some outliers
- data = {'TransactionAmount': [50, 75, 100, 45, 60, 55, 1200, 80, 90, 3000, 65]}
- df = pd.DataFrame(data)

- # Create a box plot
- plt.boxplot(df['TransactionAmount'])
- plt.title('Transaction Amount Boxplot')
- plt.ylabel('Amount')
- plt.show()
- What you'll see:
- The box plot will display most transactions clustered in a box representing the middle 50% of values. The points far above the box (like 1200 and 3000) will appear as outliers, visually highlighting unusually large transactions.

Understand Feature Relationships

- **What it means:**
  Seeing how features correlate helps you choose which are important or whether transformations are needed.
- **Example:**
  A scatter plot between Age and Fare can show if older passengers tend to pay higher fares

Choose Feature Transformations

- **What it means:**
  Visualization can show if data needs scaling, normalization, or log transformation to improve model performance.
- **Example:**

If Fare is heavily skewed, a histogram of log-transformed Fare can reveal better distribution.

Libraries Used for Visualization

.1 Matplotlib

Matplotlib is a **low-level plotting library** in Python.
It gives **full control** over graphs.

Used for:

- Simple plots
- Custom visualizations
- Foundational plotting

Seaborn

Seaborn is a **high-level visualization library** built on top of Matplotlib.
It produces **beautiful and informative plots with less code**.

Used for:

- Statistical plots
- ML data exploration
- Correlation analysis

Importing Libraries

```python
import matplotlib.pyplot as plt
import seaborn as sns
```

Explanation plt
→ matplotlib plotting functionssns
→ seaborn plotting functions

5. Basic Plot Structure (IMPORTANT)

General Syntax

```python
plt.plot(x, y)
plt.show()
```

Explanation

- `plot()` → creates graph
- `show()` → displays graph

6. Line Plot

Definition

A line plot shows **change over time or sequence.**

Syntax

```python
plt.plot(x, y)
```

ML Meaning

- Used for loss vs epochs
- Learning curves

```python
x = [1, 2, 3, 4]
y = [10, 20, 15, 25]

plt.plot(x, y)
plt.xlabel("X values")
plt.ylabel("Y values")
plt.title("Line Plot Example")
plt.show()
```

7. Bar Plot

Definition

Bar plot shows **comparison between categories.**

Syntax

```python
plt.bar(categories, values)
```

ML Use

- Class distribution
- Categorical feature analysis

Example

```python
names = ['A', 'B', 'C']
scores = [80, 70, 90]

plt.bar(names, scores)
plt.show()
```

8. Histogram (VERY IMPORTANT)

Definition

Histogram shows **distribution of numerical data.**

Syntax

```python
plt.hist(data, bins)
```

ML Meaning

- Detect skewness
- Decide mean vs median
- Check normal distribution

Example

```python
ages = [20, 22, 25, 30, 35, 40, 22, 25]

plt.hist(ages, bins=5)
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

9. Box Plot

Definition

Box plot shows:

- Median
- Quartiles
- Outliers

Syntax

```python
plt.boxplot(data)
```

ML Meaning

- Detect outliers
- Decide outlier treatment

Example

```python
plt.boxplot(ages)
plt.show()
```

10. Scatter Plot

Definition

Scatter plot shows **relationship between two numerical variables.**

Syntax

```python
plt.scatter(x, y)
```

ML Meaning

- Feature correlation
- Linear relationship detection

Example

```python
plt.scatter(df['Age'], df['Salary'])
plt.xlabel("Age")
plt.ylabel("Salary")
plt.show()
```

Question

- Using a histogram graphical representation that shows the distribution of a numerical dataset by grouping data points into bins (intervals) and displaying the count of data points in each bin as bars, show how to create a histogram using Matplotlib with the Iris dataset's sepal_length column